

VS Code + Apple Clang/LLDB : C 语言环境配置指南 (macOS)

适用对象：第一次接触 C 语言程序设计的本科一年级同学
本文以课程工作文件夹 `~/Documents/C_Course/c` 为例。
目标：能够在 VS Code 中编辑、编译、运行和调试一个 `.c` 程序。

1. 先认识三个不同的工具

安装 VS Code 并不等于安装了 C 语言编译器。

工具	作用	本课程采用
VS Code	编辑源代码	Visual Studio Code
C/C++ 扩展	提供代码补全、错误提示和调试入口	Microsoft C/C++
编译器和调试器	把 <code>.c</code> 文件编译为可执行文件，并支持断点调试	Apple Clang + LLDB

可以把它们理解为：VS Code 是“书桌”，C/C++ 扩展是“辅助工具”，Clang 才是把源代码变成可执行程序“编译器”，LLDB 负责暂停程序、单步执行和查看变量。

macOS 自带的 `/usr/bin/gcc` 实际上通常也是 Apple Clang 的入口。为避免混淆，本文统一使用命令 `clang`。

2. 安装 VS Code 和 C/C++ 扩展

1. 从 [VS Code 官方网站](#) 下载 macOS 版 VS Code。
2. 打开下载的压缩包或磁盘映像，把 `Visual Studio Code.app` 拖入“应用程序”文件夹，然后启动。
3. 按 `⇧⌘X` (Shift+Command+X) 打开“扩展”。
4. 搜索 `C/C++`。
5. 安装发布者为 Microsoft、扩展 ID 为 `ms-vscode.cpptools` 的扩展。

VS Code 下载页可能按电脑型号提供 Apple silicon、Intel chip 或 Universal 版本。单击屏幕左上角的苹果菜单，选择“关于本机”，即可查看芯片类型；不确定时可选择 Universal 版本。

本课程不依赖 Code Runner 扩展。初学阶段建议使用本文的 Clang 编译任务，避免不同扩展的“运行”按钮造成混淆。

3. 安装 Apple Command Line Tools

Clang 和 LLDB 由 Apple 的 Command Line Tools for Xcode 提供。只学习命令行 C 程序时，无需下载完整的 Xcode。

1. 打开 macOS 的“终端”应用。
2. 执行：

```
xcode-select --install
```

3. 系统弹出安装窗口后，单击“安装”，阅读并同意许可协议。
4. 等待下载和安装完成。

如果终端提示命令行工具已经安装，可以直接进入下一节。若电脑已经安装完整 Xcode，也不需要重复安装 Command Line Tools。

Command Line Tools 通常安装在：

```
/Library/Developer/CommandLineTools
```

其中包含 macOS SDK、Clang、LLDB 和其他开发工具。与 Windows 上的 MinGW-w64 不同，它们由系统工具 `xcrun` 按当前开发者目录定位，一般不需要手工下载工具链压缩包。

4. 检查编译器、调试器和 PATH

在一个新打开的 macOS 终端或 VS Code 终端中依次输入：

```
clang --version
lldb --version
xcrun --find clang
xcrun --find lldb
xcode-select -p
```

正确现象：前两条命令显示版本；`xcrun --find` 显示 Clang 和 LLDB 的实际位置；最后一条显示当前开发者目录。

常见输出路径类似：

```
/Library/Developer/CommandLineTools/usr/bin/clang
/Library/Developer/CommandLineTools/usr/bin/lldb
/Library/Developer/CommandLineTools
```

macOS 为什么通常不用手工修改 PATH

macOS 默认的 PATH 已经包含 `/usr/bin`，而 `/usr/bin/clang` 会转到当前选中的 Apple 工具链。因此本文的 VS Code 配置直接使用：

```
/usr/bin/clang
```

可以用下面的命令再确认一次：

```
which clang
```

如果提示需要安装开发者工具，先完成第 3 节；如果提示找不到有效的开发者目录，参见第 10 节的问题排查。

5. 认识本课程的示例工作文件夹

解压双系统包后，请在 VS Code 中直接打开 `macOS` 文件夹；也可以将整个 `macOS` 文件夹复制并重命名为 `c`，放到下面的示例位置。不要打开包含两个系统文件夹的上一层目录。

示例工作区为：

```
~/Documents/C_Course/c
├── .vscode
│   ├── c_cpp_properties.json
│   ├── launch.json
│   ├── settings.json
│   └── tasks.json
└── C1
    └── A.c
```

各部分含义如下：

- `~/Documents/C_Course/c`：工作区根目录。打开 VS Code 时应打开这个文件夹，而不是只打开某个 `A.c` 文件。
- `.vscode`：只对当前工作区生效的配置目录。
- `tasks.json`：告诉 VS Code 如何调用 Clang 编译当前 C 文件。
- `launch.json`：告诉 VS Code 如何运行或使用 LLDB 调试编译出来的程序。
- `c_cpp_properties.json`：告诉 C/C++ 扩展编译器位置、C 标准和代码补全设置。
- `settings.json`：保存该工作区的编辑器设置。
- `C1/A.c`：本次上机的 C 源代码。

请在 VS Code 中选择“文件 → 打开文件夹”，打开整个 `c` 文件夹。左侧资源管理器应同时看到 `.vscode` 和 `C1`。

`.vscode` 是工作区配置，所以它可以服务于 `C1` 以及将来建立的其他上机子文件夹。

6. 配置当前工作区

下面四份文件放在工作区根目录的 `.vscode` 文件夹中。macOS 路径使用正斜杠 `/`，生成的可执行文件通常没有 `.exe` 后缀。

6.1 .vscode/c_cpp_properties.json

```
{
  "configurations": [
    {
      "name": "macOS Clang",
      "includePath": ["${workspaceFolder}/**"],
      "defines": [],
      "compilerPath": "/usr/bin/clang",
      "cStandard": "c11",
      "cppStandard": "c++17"
    }
  ],
  "version": 4
}
```

关键项：

- `compilerPath`：Clang 的入口位置。扩展会向编译器查询系统头文件和目标架构。
- `cStandard: "c11"`：本课程按 C11 标准检查代码。
- `includePath`：当前工作区内头文件的搜索范围；标准库头文件路径不需要手工填写。

这里没有写死 `intelliSenseMode`，因此同一份配置可以让扩展根据本机编译器自动识别 Apple 芯片的 `arm64` 或 Intel Mac 的 `x86_64`。

6.2 .vscode/tasks.json

```
{
  "version": "2.0.0",
  "tasks": [
    {
      "type": "cppbuild",
      "label": "C: build release",
      "command": "/usr/bin/clang",
      "args": [
        "-std=c11", "-Wall", "-Wextra", "-pedantic", "-O2", "-fdiagnostics-color=always", "${file}",
        "-o", "${fileDirname}/${fileBasenameNoExtension}"
      ],
      "options": {
        "cwd": "${fileDirname}"
      },
      "problemMatcher": ["$gcc"],
      "group": {
        "kind": "build",
        "isDefault": true
      },
      "detail": "使用 Apple Clang 以 C11 模式编译当前 C 文件（发布版）"
    },
    {
      "type": "cppbuild",
      "label": "C: build debug",
      "command": "/usr/bin/clang",
      "args": [
        "-std=c11", "-Wall", "-Wextra", "-pedantic", "-O0", "-g3", "-fdiagnostics-color=always",
        "${file}", "-o", "${fileDirname}/${fileBasenameNoExtension}"
      ],
      "options": {
        "cwd": "${fileDirname}"
      },
      "problemMatcher": ["$gcc"],
      "group": {
        "kind": "build",
        "isDefault": false
      },
      "detail": "使用 Apple Clang 以 C11 模式编译当前 C 文件（调试版）"
    }
  ]
}
```

几个常见编译参数：

参数	含义
-std=c11	使用 C11 标准
-Wall -Wextra -pedantic	尽量显示有教学价值的警告
-O2	优化发布版程序
-O0	调试时关闭优化，使单步执行更直观
-g3	把调试信息写入程序，供 LLDB 使用
-o ...	指定输出的可执行文件名和位置

`${file}` 表示当前编辑器中正在打开的文件。因此编译前应先单击 [A.c](#)，使它成为当前文件。

6.3 .vscode/launch.json

```
{
  "version": "0.2.0",
  "configurations": [
    {
      "name": "运行当前 C 文件",
      "type": "cppdbg",
      "request": "launch",
      "program": "${fileDirname}/${fileBasenameNoExtension}",
      "args": [],
      "stopAtEntry": false,
      "cwd": "${fileDirname}",
      "environment": [],
      "externalConsole": true,
      "MIMode": "lldb",
      "preLaunchTask": "C: build release"
    },
    {
      "name": "调试当前 C 文件",
      "type": "cppdbg",
      "request": "launch",
      "program": "${fileDirname}/${fileBasenameNoExtension}",
      "args": [],
      "stopAtEntry": false,
      "cwd": "${fileDirname}",
      "environment": [],
      "externalConsole": true,
      "MIMode": "lldb",
      "preLaunchTask": "C: build debug"
    }
  ]
}
```

注意：preLaunchTask 的文字必须与 tasks.json 中对应任务的 label 完全相同。

externalConsole: true 会让程序在 macOS “终端” 窗口中运行，便于使用 scanf 输入。第一次启动时，系统可能询问是否允许 VS Code 控制“终端”，请选择允许；相关权限可在“系统设置 → 隐私与安全性 → 自动化”中检查。

6.4 .vscode/settings.json（可选）

```
{
  "files.defaultLanguage": "c",
  "editor.formatOnType": true,
  "editor.snippetSuggestions": "top",
  "files.encoding": "utf8"
}
```

7. 编写第一个程序

当前 `C1/A.c` 已有一个最小的 C 程序框架。为了看到明显的运行结果，可以暂时写成：

```
#include <stdio.h>

int main(void)
{
    printf("Hello, C!\n");
    return 0;
}
```

按 `⌘S` 保存。

这段程序在做什么

- `#include <stdio.h>`：使用标准输入输出库。
- `int main(void)`：程序从 `main` 函数开始执行。
- `printf(...)`：在屏幕上输出文字。
- `\n`：换行。
- `return 0;`：告诉操作系统程序正常结束。

8. 编译和运行

方式一：先在终端中操作（建议第一次课先练习）

在 VS Code 中按 `⌘``（Control+反引号）打开终端。确认终端当前位于工作区根目录，再执行：

```
cd "C1"
clang A.c -std=c11 -Wall -Wextra -pedantic -o A
./A
```

如果看到：

```
Hello, C!
```

说明“源代码 → Clang 编译 → macOS 运行”这一完整流程已经成功。

其中：

```
A.c  --Clang 编译-->  A  --macOS 运行-->  输出结果
```

`./A` 中的 `./` 表示运行当前目录中的文件。修改源代码后，必须重新编译，旧的可执行文件 `A` 不会自动变化。

方式二：使用 VS Code 编译并运行（推荐日常使用）

1. 在编辑器中单击打开 `A.c`，使它成为当前文件。
2. 按 `^F5`，即“运行但不调试”。部分 Mac 需要同时按 `fn`。
3. 如果第一次运行时出现配置选择，请选择“运行当前 C 文件”。
4. VS Code 会先调用 Clang 编译 `A.c`，生成可执行文件 `A`，然后自动运行它。
5. 在弹出的“终端”窗口中查看程序输出；如果程序要求输入，就在该窗口中输入。

简单原理如下：

```

^F5
↓
读取 launch.json 中的“运行当前 C 文件”
↓
根据 preLaunchTask 调用 tasks.json 中的“C: build release”
↓
Clang 把当前 A.c 编译成 A
↓
VS Code 直接运行 A，但不在断点处暂停

```

也可以按 `⇧⌘B` 执行默认编译任务，再在工作区根目录的终端中输入 `./C1/A` 运行；若已经进入 `C1` 目录，则输入 `./A`。

判断编译是否成功，应看终端中的 Clang 输出，不应只看代码下方有没有红色波浪线。如果编译失败，程序不会进入运行阶段，应先修改终端中报告的第一个错误。

9. 断点调试

调试可以暂停程序，逐行观察变量变化。

1. 单击打开 `A.c`。
2. 在某一行左侧行号栏单击，出现红点，这就是断点。
3. 打开左侧“运行和调试”（`⇧⌘D`）。
4. 在顶部选择“调试当前 C 文件”。
5. 按 `F5`。VS Code 会先执行 `C: build debug`，再由 LLDB 启动程序。
6. 程序停在断点后，可使用：
 - `F10`：单步跳过（执行当前行，不进入函数内部）；
 - `F11`：单步进入函数；
 - `⇧F11`：跳出当前函数；
 - `F5`：继续运行到下一个断点。

Mac 键盘若把 `F5`、`F10`、`F11` 用作亮度或系统功能键，请同时按住 `fn`。如果程序需要用 `scanf` 输入，请在弹出的“终端”窗口中输入。

10. 常见问题排查

问题 1：执行 `xcode-select --install` 时提示已经安装

这通常不是错误。执行以下命令检查：

```
xcode-select -p
xcrun --find clang
clang --version
```

三条命令都有正常输出即可继续。

问题 2：提示 `xcrun: error: invalid active developer path`

先重新运行：

```
xcode-select --install
```

安装完成后关闭并重新打开终端。若安装了完整 Xcode，可在 Xcode 的设置中选择 Command Line Tools，或按 Apple 官方说明切换当前开发者目录。

问题 3：`#include <stdio.h>` 下方出现红色波浪线

1. 先确认终端中的 `clang --version` 成功；
2. 检查 `c_cpp_properties.json` 的 `compilerPath` 是否为 `/usr/bin/clang`；
3. 按 `⌘P`，运行 `C/C++: Select IntelliSense Configuration`，选择 Clang；
4. 再运行 `Developer: Reload Window` 重新载入窗口。

红色波浪线来自 IntelliSense，真正的编译结果来自 Clang；两者配置相关，但不是同一个程序。

问题 4：按 `F5` 后 LLDB 或外部“终端”没有启动

1. 检查 Microsoft C/C++ 扩展是否已启用；
2. 执行 `xcrun --find lldb` 和 `lldb --version`；
3. 在“系统设置 → 隐私与安全性 → 自动化”中允许 Visual Studio Code 控制“终端”；
4. 完全退出 VS Code 后重新打开工作区。

如果只需要验证程序，可先用第 8 节的终端命令 `./A` 运行。

问题 5：提示找不到 `preLaunchTask`

检查 `launch.json` 的 `preLaunchTask` 和 `tasks.json` 的 `label` 是否逐字相同，并确认 `tasks.json` 是有效 JSON。

问题 6：按 F5 后提示找不到程序 A

通常是编译步骤已经失败。向上查看终端中最早出现的 Clang 错误，先修改源代码，再重新按 F5。

问题 7：终端输入 A 后提示 command not found

macOS 默认不会在当前目录搜索程序。请使用：

```
./A
```

如果提示 `permission denied`，先确认 A 确实由 Clang 编译生成，再执行 `ls -l A` 检查它是否具有执行权限。

问题 8：Apple 芯片与 Intel Mac 的配置不同吗

本文使用 `/usr/bin/clang`，由系统自动选择适合当前电脑的目标架构。可用 `uname -m` 查看：Apple 芯片通常显示 `arm64`，Intel Mac 显示 `x86_64`。本文没有在 JSON 中写死架构，所以两类电脑可使用同一份配置。

问题 9：修改了一个文件，却编译了另一个文件

本教程的任务使用 `${file}`，只编译当前活动文件。按 `⌘B` 或 F5 前，先单击要编译的 .c 文件标签页。

问题 10：一个程序有多个 .c 文件怎么办

本教程统一使用英文子目录 C1，命令中的目录名应与实际文件夹一致。

本教程的配置适合“一个 .c 文件就是一个程序”的入门阶段。以后学习多文件程序时，需要在 Clang 命令中列出所有源文件，或使用 Make/CMake；不要简单地给每个文件都写一个 `main` 函数。

11. 课前自检清单

上课前请逐项确认：

- ☐ VS Code 可以正常启动；
- ☐ 已安装 Microsoft C/C++ 扩展；
- ☐ `clang --version` 有版本输出；
- ☐ `lldb --version` 有版本输出；
- ☐ `xcode-select -p` 显示有效的开发者目录；
- ☐ 使用 VS Code 打开的是课程文件夹，而不是单独的 `.c` 文件；
- ☐ `.vscode` 中的编译器路径为 `/usr/bin/clang`；
- ☐  能生成没有 `.exe` 后缀的可执行文件；
- ☐ 终端可以用 `./A` 运行程序并看到输出；
- ☐ `F5` 可以在断点处暂停。

完成以上项目，macOS 上的 C 语言实验环境就配置好了。

参考资料

- [VS Code 官方：Using Clang in Visual Studio Code](#)
- [VS Code 官方：Configure C/C++ IntelliSense](#)
- [VS Code 官方：Configure C/C++ debugging](#)
- [Apple 官方：Installing the command-line tools](#)